

به نام خداوند جان و خرد

آزمایش ششم آزمایشگاه طراحی سیستم‌های دیجیتال

اعضای گروه

پوریا رحمانی ۴۰۲۱۱۱۴۱۸

امیرمحمد شربتی ۴۰۲۱۰۶۱۱۲

منطق طراحی

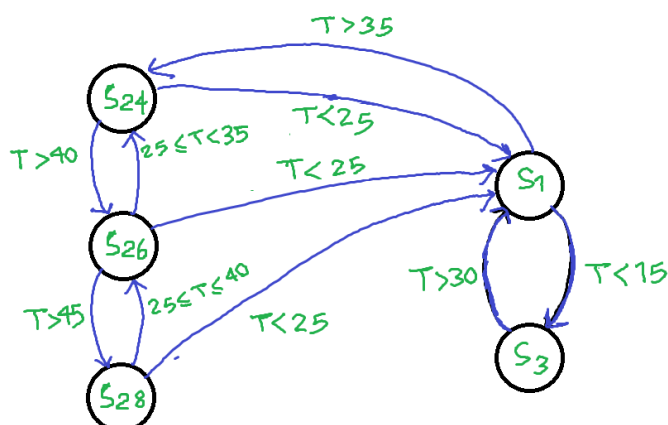
برای طراحی مدار، ابتدا دو نمودار حالت را به یک نمودار حالت تبدیل می‌کنیم. به این صورت که یک نمودار حالت با 5 حالت در نظر می‌گیریم که S1 و S3 در آن مشابه نمودار حالت سمت چپ در صورت سوال باشند و به جای S2 سه حالت به نام‌های S24 (کولر روشن و دور آن برابر 4 است)، S26 (کولر روشن و دور آن برابر 6 است) و S28 (کولر روشن و دور آن برابر 8 است) در نظر می‌گیریم.

Transitoion ها بین حالات S1 و S3 مشابه نمودار حالت سمت چپ شکل صورت سوال است. اگر در S1 باشیم و دما بیشتر از 35 درجه باشد به حالت S24 می‌رویم (یعنی کولر را روشن می‌کنیم و دور آن را روی 4 می‌گذاریم، همانطور که نمودار سمت چپ می‌گوید باید کولر را روشن کنید و نمودار سمت راست می‌گوید از حالت Out باید به حالتی بروید که دور کولر برابر 4 است).

اگر در هر کدام از S24، S26، یا S28 باشیم و دما کمتر از 25 درجه شود، همانطور که از نمودار سمت چپ معلوم است، باید کولر را خاموش کنیم و در نتیجه به حالت S1 برویم.

اگر در S24 بودیم و دما بیشتر از 40 درجه بود، طبق نمودار سمت راست باید به S26 برویم و اگر در S26 بودیم و دما بیشتر از 45 درجه بود، باید به S28 برویم.

از طرفی، اگر در S26 بودیم و دما بیشتر یا مساوی 25 درجه و کمتر از 35 درجه بود، طبق نمودار سمت چپ باید کولر روشن بماند و طبق نمودار سمت راست باید دور کولر را به 4 کاهش دهیم، بنابراین باید به S24 برویم. اگر هم در S28 بودیم و دما بیشتر یا مساوی 25 درجه و کمتر از 40 درجه بود، طبق نمودار سمت چپ باید کولر روشن بماند و طبق نمودار سمت راست، باید دور کولر را به 6 کاهش دهیم. با توجه به این توضیحات، نمودار حالت ما مطابق شکل زیر خواهد شد.



پیاده‌سازی با verilog

مدار باید یک ورودی **reset** داشته باشد (آن را **active low** قرار می‌دهیم) که وضعیت مدار را به حالت اولیه یعنی **S1** ببرد و یک ورودی **۸ بیتی** که نمایش **علامت‌دار به صورت مکمل دو** ما را به ما نشان دهد.

از طرفی مدار یک خروجی یک بیتی دارد که روشن یا خاموش بودن **heater** را به ما نشان می‌دهد و یک خروجی چهار بیتی که دور کولر را به ما نشان می‌دهد. (اگر کولر خاموش باشد، عدد صفر را نشان خواهد داد و بنابراین نیاز به خروجی جدا برای روشن بودن یا نبود کولر نداریم).

از طرفی از آنجا که نمودار حالت ما ۵ حالت دارد، یک **reg** کمکی سه بیتی برای نمایش شماره حالت نگه می‌داریم که صفر است اگر در **S1** باشیم، یک است اگر در **S24** باشیم، دو اگر در **S26**، سه اگر در **S28** و چهار اگر در **S3** باشیم. بعلاوه از آنجا که گفته شده است دما هر یک دقیقه یک‌بار خوانده می‌شود، مدار باید ترتیبی باشد و یک سیگنال **Clk** داشته باشد تا در لبه بالا رونده آن (در شرایطی که در حالت **reset** نیستیم) محاسبات و جابجایی‌ها بین حالات انجام شود. (اگر در آزمایشگاه از ما خواسته شد که مدار را به پالس ساعت خود برد وصل کنیم و کران یک دقیقه را هم رعایت کنیم، با توجه به فرکانس پالس ساعت، یک **reg** دیگر تعریف خواهیم کرد تا در هر تعداد مشخص از چرخه ساعت یک‌بار (طوری‌که معادل یک دقیقه شود) محاسبات انجام شود و کد نهایی را در گزارش کار خواهیم آورد)

با توجه به این موارد و نمودار حالت، کد را پیاده‌سازی می‌کنیم.

تصویر ورودی‌ها، خروجی‌ها و **reg** استفاده شده برای **state**:

```
module Incubator (  
    input wire RstN,  
    input wire Clk,  
    input wire signed [7:0] temp,  
    output reg is_heater_on,  
    output reg [3:0] cooler_RPS  
);  
  
reg [2:0] state;
```

مابقی کد یک دنباله از **if** و **else if** هاست که کاملاً مطابق نمودار حالت جدیدی که رسم کردیم، مشخص می‌کند در هر **state** و بازه دمایی مشخص، باید به چه **state** ای برویم و در وضعیت **RPS** کولر و روشن یا خاموش بودن **heater** چه تغییری رخ می‌دهد. عکس قسمتی از کد را در زیر می‌بینید. نمونه کامل کد به پیوست ارسال شده است. توجه کنید برای هماهنگی سخت‌افزار با رفتار **verilog** در کد رفتاری که زدیم از **non-blocking assignment** استفاده کردیم. (اگر از **blocking assignment** استفاده می‌کردیم، به کد یک فرض **sequential** بودن اجرای دستورات وارد می‌شد که در سخت‌افزار از آنجا که همه تغییرات در لبه **clock** اتفاق می‌افتند چنین شرطی برقرار نیست).

همینطور می‌توانستیم از ساختار switch-case برای عدد state و بعد درون آن از if-else برای حالت بندی روی دما استفاده کنیم اما از دوستانی که ترم‌های پیش این درس را پاس کردند شنیدیم که ساختارهای if تو در تو در سنتز ممکن است دچار مشکل شوند.

تصویر بخشی از کد

```
always @(posedge Clk, negedge RstN) begin
    if(!RstN) begin
        state <= 0;
        is_heater_on <= 0;
        cooler_RPS <= 0;
    end

    else if(state == 3'd0 && temp < 8'sd15) begin
        state <= 3'd4;
        is_heater_on <= 1;
    end

    else if(state == 3'd0 && temp > 8'sd35) begin
        cooler_RPS <= 4'd4;
        state <= 3'd1;
    end

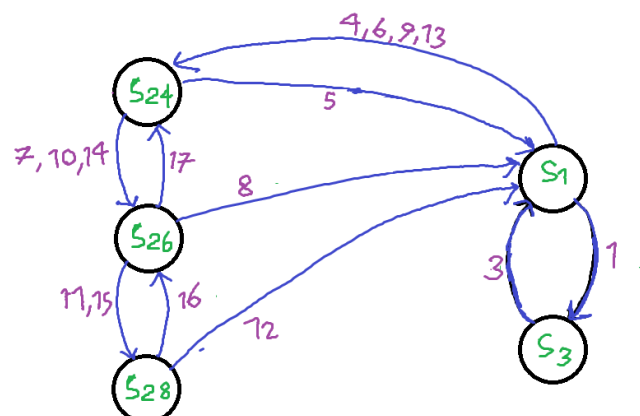
    else if(state == 3'd1 && temp < 8'sd25) begin
        cooler_RPS <= 4'd0;
        state <= 3'd0;
    end

    else if(state == 3'd1 && temp > 8'sd40) begin
        cooler_RPS <= 4'd6;
        state <= 3'd2;
    end

    else if(state == 3'd2 && temp > 8'sd45) begin
        cooler_RPS <= 4'd8;
        state <= 3'd3;
    end
end
```

نوشتن testbench

برای تست‌بنچ تلاش کردیم طوری دما را جابجا کنیم که تمام یال‌های نمودار حالتی که رسم کردیم cover شوند تا تست coverage بالایی داشته‌باشد. در شکل زیر ترتیب طی کردن یال‌ها توسط testbench را می‌بینید. (کنار هر یال، شماره گام‌هایی از تغییر دما که آن را طی کردیم را نوشته‌ایم)



برای آنکه از تکرار کد جلوگیری شود از ساختار task استفاده کردیم. ابتدا task هایی برای چاپ کردن وضعیت heater و RPS کولر نوشتیم:

```
task show_cooler_state(input [3:0] cooler_round_per_second);
begin
    case(cooler_round_per_second)
        4'd0: $display("Cooler is off!");
        4'd4: $display("Cooler is on and RPS = 4");
        4'd6: $display("Cooler is on and RPS = 6");
        4'd8: $display("Cooler is on and RPS = 8");
    endcase
end
endtask

task show_heater_state(input heater_state);
begin
    case(heater_state)
        1'd0: $display("Heater is off!");
        1'd1: $display("Heater is on!");
    endcase
end
endtask
```

سپس از این task ها استفاده کردیم و یک task نوشتیم تا دما را عوض کرده و حالت cooler و heater را در اثر این تغییر چاپ کرده و اگر نتیجه درست بود بنویسد Test Passed! و در غیر این صورت بنویسد Task Failed!. برای این کار دما و مقدار درست RPS کولر و وضعیت heater را به task ورودی دادیم، ورودی دما را در temp (reg) ای که به ورودی دمای نمونه Incubator ما وصل است) ریختیم، تا لبه بالارونده clock بعدی صبر کردیم و سپس RPS کولر و وضعیت Incubator را چاپ کرده و اگر با مقادیر ورودی تابع برابر بودند (یعنی درست بودند) عبارت Test Passed! وگرنه عبارت Test Failed! چاپ شد.

ابتدا RstN را برابر صفر گذاشتیم تا بررسی کنیم reset به درستی انجام می شود. سپس، برای این که دنباله ای از یال ها که می خواهیم طی شوند این دنباله از تغییر دما ها را انجام دادیم.

```
task set_temp_and_show_stat(input signed [7:0] temperature, input correct_heater, input [3:0] correct_cooler);
begin
    RstN = 1;
    temp = temperature;
    @(posedge Clk);
    #1;
    show_cooler_state(cooler_RPS);
    show_heater_state(is_heater_on);
    if(is_heater_on == correct_heater && cooler_RPS == correct_cooler) $display("Test Passed!");
    else begin
        $display("Test Failed!");
        failed = failed + 1;
    end
    $display("-----\n");
    test_number = test_number + 1;
end
endtask
```

۱۲ (روشن شدن heater) ← منفی ۱۰ (روشن ماندن heater) ← ۳۱ (خاموش کردن heater) ← ۳۶ (روشن کردن کولر با RPS = ۴) ← ۲۴ (خاموش کردن کولر) ← ۴۱ (روشن کردن کولر با RPS = ۴) ← ۴۱ (افزایش RPS به ۶) ← ۲۲ (خاموش کردن کولر) ← ۴۶ (روشن کردن کولر با RPS = ۴) ← ۴۶ (افزایش RPS به ۶) ← ۴۶ (افزایش RPS به ۸) ← منفی ۱ (خاموش کردن کولر) ← ۵۰ (روشن کردن کولر با RPS = ۴) ← ۵۰ (افزایش RPS به ۶) ← ۵۰ (افزایش RPS به ۸) ← ۳۵ (کاهش RPS به ۶) ← ۳۰ (کاهش RPS به ۴)

تصویر gtkwave که از شبیه‌سازی کوارتوس (با modelsim altera) بدست آمد:

/tb/Ck	1																		
/tb/RstN	1																		
/tb/temp	30	22	46																
/tb/is_heater_on	510							1		50						35		30	
/tb/cooler_RPS	0100	0...	0000		0100		0110		1000		0000		0100		0110		1000		0110
/tb/failed	00000	00000																	
/tb/test_number	10001	0...	01000		01001		01010		01011		01100		01101		01110		01111		10000
																			10001

(یک reg برای تعداد تست‌ها و یک reg برای تعداد تست‌های fail شده هم گذاشتیم که اگر تست موفقیت آمیز نبود تعداد تست‌های fail شده را نمایش بدهد و اگر موفقیت آمیز بود(که قاعدتا برای کد ما بود!) نشان بدهد که همه تست‌ها PASS شده‌اند.)

در تصاویر زیر نتیجه تست‌بنچ را می‌بینید. تصاویر را از چپ به راست ببینید.(کد تست‌بنچ نیز به ضمیمه ارسال شده‌است):

```
[Running] Incubator.v
VCD info: dumpfile wave.vcd opened for output.
Cooler is off!
Heater is off!
Reset was successful!
-----
Making the weather cold!
Cooler is off!
Heater is on!
Test Passed!
-----
Making the weather colder! You shouldn't change anything.
Cooler is off!
Heater is on!
Test Passed!
-----
Got warm! nothing should be on!
Cooler is off!
Heater is off!
Test Passed!
-----
Got warmer! Turn the cooler on with RPS = 4.
Cooler is on and RPS = 4
Heater is off!
Test Passed!
-----
Got cold! Turn the cooler of.
Cooler is off!
Heater is off!
Test Passed!
-----
Got warm again. turn the cooler on with RPS = 4
Cooler is on and RPS = 4
Heater is off!
Test Passed!
-----
Still warm! increase the RPS to 6.
Cooler is on and RPS = 6
Heater is off!
Test Passed!
-----
Suddenly, got too cold! Turn the cooler off.
Cooler is off!
Heater is off!
Test Passed!
-----
Got warm! turn the cooler on again with RPS = 4
Cooler is on and RPS = 4
Heater is off!
Test Passed!
-----
Still too warm! increase the RPS to 6
Cooler is on and RPS = 6
Heater is off!
Test Passed!
-----
Still too warm! increase the RPS to 8
Cooler is on and RPS = 8
Heater is off!
Test Passed!
-----
Suddenly, it Got cold. Turn the cooler of!
Cooler is off!
Heater is off!
Test Passed!
-----
Got warm fot the fourth time!! turn the cooler on with RPS = 4.
Cooler is on and RPS = 4
Heater is off!
Test Passed!
-----
Still warm! increase RPS to 6.
Cooler is on and RPS = 6
Heater is off!
Test Passed!
-----
Still warm! increase RPS to 8.
Cooler is on and RPS = 8
Heater is off!
Test Passed!
-----
Better but still warm. decrease RPS to 6
Cooler is on and RPS = 6
Heater is off!
Test Passed!
-----
Better but still warm. decrease RPS to 4
Cooler is on and RPS = 4
Heater is off!
Test Passed!
-----
RESULT: PASSED
Incubator.v:210: $finish called at 941 (1s)
```